ROBOTIC SYSYTEMS [UE21EC342BC3]

"Turtlebot3 simulation using Ros noetic and Gazebo and Density based Traffic control using Arduino and ultrasonic sensor"

Submitted By :-

Charan B K [PES1UG21EC075]

Darshan Raj G [PES1UG21EC079]

Under the guidance of

Dr. Shikha Tripathi

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

PES University Bengaluru - 560085

CONTENTS :-

1) Problem statement .

2) Code.

3) Results.

4) Conclusion.

5) References.

1.A) Problem statement :- " Turtlebot3 simulation using Ros noetic and Gazebo"

- Steps involved:-
  - Install simulation package.
  - Launch simulation world.
  - Operate TurtleBot3.

- Code:-

```
#include "turtlebot3_gazebo/turtlebot3_drive.h"
```

```
Turtlebot3Drive::Turtlebot3Drive()
: nh_priv_("~")
{
//Init gazebo ros turtlebot3 node
ROS_INFO("TurtleBot3 Simulation Node Init"); auto
ret = init();
ROS_ASSERT(ret);
}

Turtlebot3Drive::~Turtlebot3Drive()
{ updatecommandVelocity(0.0,
0.0); ros::shutdown();
}
```

* Init function

```
bool Turtlebot3Drive::init()
{
// initialize ROS parameter std::string cmd_vel_topic_name =
nh_.param<std::string>("cmd_vel_topic_name", "");

// initialize variables
escape_range_       = 30.0 * DEG2RAD;
check_forward_dist_ = 0.7; check_side_dist_
= 0.6;

tb3_pose_ = 0.0; prev_tb3_pose_
= 0.0;

// initialize publishers cmd_vel_pub_   =
nh_.advertise<geometry_msgs::Twist>(cmd_vel_topic_name, 10);
```

```
// initialize subscribers laser_scan_sub_ = nh_.subscribe("scan", 10,
&Turtlebot3Drive::laserScanMsgCallBack, this); odom_sub_ = nh_.subscribe("odom", 10,
&Turtlebot3Drive::odomMsgCallBack, this);

return true;
}

void Turtlebot3Drive::odomMsgCallBack(const nav_msgs::Odometry::ConstPtr &msg)
{ double siny = 2.0 * (msg->pose.pose.orientation.w * msg->pose.pose.orientation.z +
msg-
>pose.pose.orientation.x * msg->pose.pose.orientation.y); double cosy = 1.0 - 2.0 * (msg-
>pose.pose.orientation.y * msg->pose.pose.orientation.y + msg>pose.pose.orientation.z * msg-
>pose.pose.orientation.z);

tb3_pose_ = atan2(siny, cosy);
}

void Turtlebot3Drive::laserScanMsgCallBack(const sensor_msgs::LaserScan::ConstPtr &msg)
{
uint16_t scan_angle[3] = {0, 30, 330};

for (int num = 0; num < 3; num++)
{ if (std::isinf(msg-
>ranges.at(scan_angle[num])))
{ scan_data_[num] = msg-
>range_max;
} else { scan_data_[num] = msg-
>ranges.at(scan_angle[num]);
}
}
}

void Turtlebot3Drive::updatecommandVelocity(double linear, double angular)
```

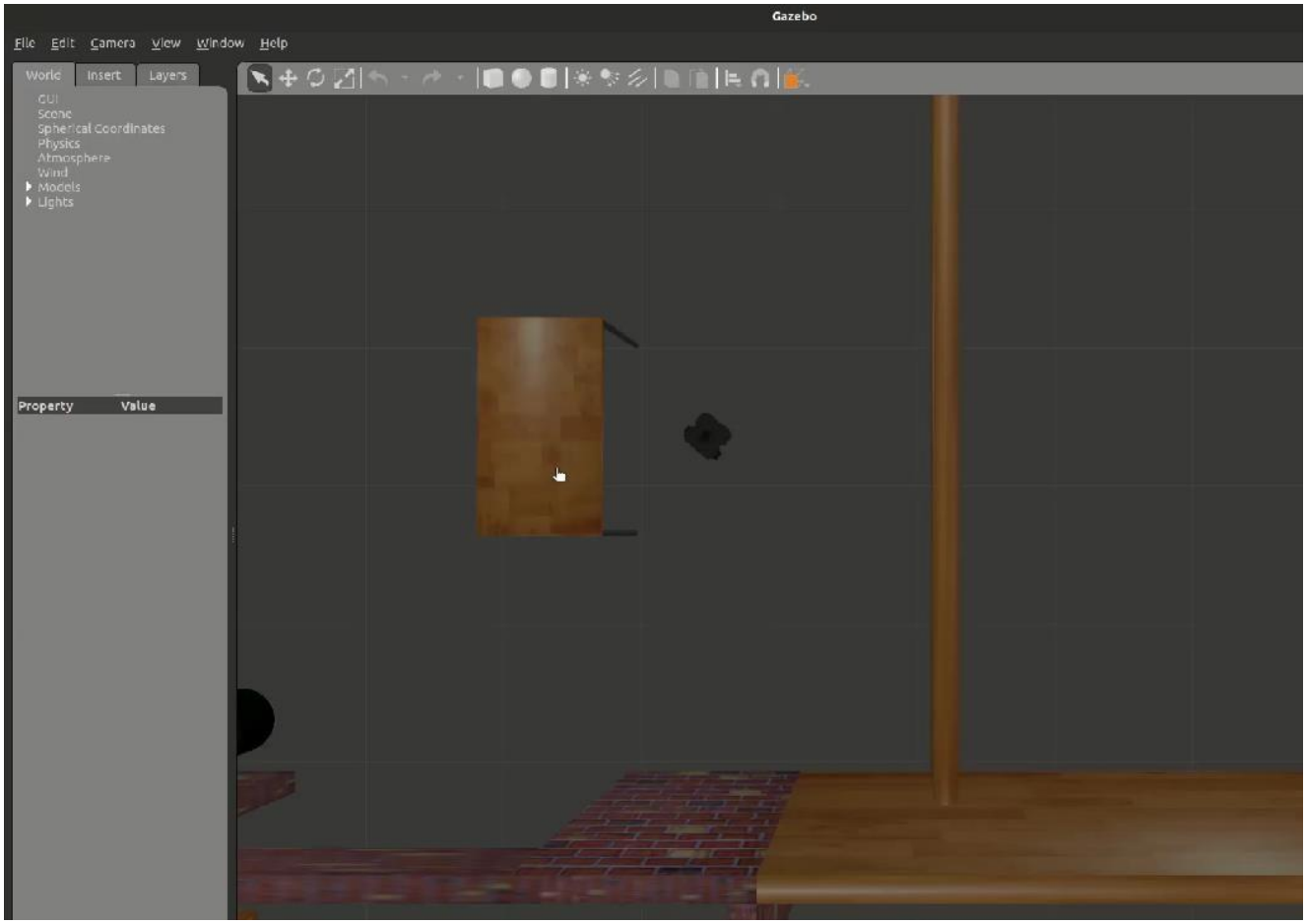
```
{ geometry_msgs::Twist
cmd_vel;

cmd_vel.linear.x  = linear; cmd_vel.angular.z
= angular;

cmd_vel_pub_.publish(cmd_vel);
}

* Control Loop function bool
Turtlebot3Drive::controlLoop()
{ static uint8_t turtlebot3_state_num =
0;

switch(turtlebot3_state_num)
{
case GET_TB3_DIRECTION:
if (scan_data_[CENTER] > check_forward_dist_)
{
if (scan_data_[LEFT] < check_side_dist_)
{ prev_tb3_pose_ = tb3_pose_;
turtlebot3_state_num = TB3_RIGHT_TURN;
} else if (scan_data_[RIGHT] <
check_side_dist_)
{ prev_tb3_pose_ = tb3_pose_;
turtlebot3_state_num = TB3_LEFT_TURN;
} else
{
turtlebot3_state_num = TB3_DRIVE_FORWARD;
}
}

if (scan_data_[CENTER] < check_forward_dist_)
```

```
{ prev_tb3_pose_ = tb3_pose_;
turtlebot3_state_num = TB3_RIGHT_TURN;
} break;

case TB3_DRIVE_FORWARD:
updatecommandVelocity(LINEAR_VELOCITY, 0.0);
turtlebot3_state_num = GET_TB3_DIRECTION; break;

case TB3_RIGHT_TURN:
if (fabs(prev_tb3_pose_ - tb3_pose_) >= escape_range_) turtlebot3_state_num
= GET_TB3_DIRECTION;
else
updatecommandVelocity(0.0, -1 * ANGULAR_VELOCITY); break;

case TB3_LEFT_TURN:
if (fabs(prev_tb3_pose_ - tb3_pose_) >= escape_range_) turtlebot3_state_num
= GET_TB3_DIRECTION;
else
updatecommandVelocity(0.0, ANGULAR_VELOCITY);
break;

default:
turtlebot3_state_num = GET_TB3_DIRECTION;
break; }

return true;
}

* Main function

int main(int argc, char* argv[])
{
ros::init(argc, argv, "turtlebot3_drive");
```

```
Turtlebot3Drive turtlebot3_drive;

ros::Rate loop_rate(125);

while (ros::ok())
{
turtlebot3_drive.controlLoop();
ros::spinOnce(); loop_rate.sleep();
}

return 0;
```

- **Results:-**

- The workspace for TurtleBot3 is set up separately using the "turtlebot3" name.
- Teleop is used to control the robot using keyboard commands
- The teleop package allows the user to control the robot using keyboard commands, with options to move forward, backward, turn left or right, and stop the robot.
- Controlling the robot using teleop and keyboard commands.



- We have attached google drive link to simulated video :-

  https://drive.google.com/drive/folders/1u_KqdOeeTbtPLrWHiv9JNJu7c9IhBT6A

1.B) Problem statement :- "Density based Traffic control using Arduino and ultrasonic sensor"

The project is aimed at designing a density based dynamic traffic signal system where the timing of signal will change automatically on sensing the traffic density at any junction. Traffic congestion is a severe problem in most cities across the world and therefore it is time to shift more manual mode or fixed timer mode to an automated system with decision making capabilities.

Present day traffic signal system is fixed time based which may render inefficient if one lane is operational than the others. To optimize this problem we have made a framework for an intelligent traffic control system. Sometimes higher traffic density at one side of the junction demands longer green time as compared to standard allotted time.

- Code :-

```
 #include "SevSeg.h"
SevSeg S;
byte CommonPins[] = {};          // common pin numbers for multi-digit display
byte SegPins[] = {2,3,4,5,6,7,8};  // 7-segment display pins in the order,{a,b,c,d,e,f,g,dp}

int red = 9;
int yellow = 10;
int green=11;

void setup()
{
  // Syntax
  // begin(COMMON_CATHODE, NumberOfDigits, CommonPins[], SevenSegPins[], resistorUsed)
  S.begin(COMMON_CATHODE, 1, CommonPins, SegPins, 1);

  pinMode(9, OUTPUT);
  pinMode(10, OUTPUT);
  pinMode(11, OUTPUT);

  digitalWrite(red, HIGH);
}

void loop()
{
  for(int i=9; i>=0; i--)
  {
    disPlay(i);
    delay(2000);
    if(i<=3)
    {
      digitalWrite(yellow, HIGH);
      digitalWrite(red, LOW);
    }
  }

  digitalWrite(green, HIGH);
  digitalWrite(yellow, LOW);

  for(int i=9; i>=0; i--)
```

```
    {
      disPlay(i);
      delay(2000);
      if(i<=3)
      {
        digitalWrite(yellow, HIGH);
        digitalWrite(green, LOW);
      }

    }

    digitalWrite(red, HIGH);
    digitalWrite(yellow, LOW);
    digitalWrite(green, LOW);



  }


  void disPlay(int num)
  {
     S.setNumber(num);
     S.refreshDisplay();
     delay(20);
  }
```

- This code utilizes a seven-segment display to simulate a traffic control system. It has been enhanced with an ultrasonic sensor to create a density-based traffic control system, adjusting the duration of the green light based on the number of cars detected.

- ## Modified code:-

```
#include "SevSeg.h"

SevSeg S;
byte CommonPins[] = {};          // common pin numbers for multi-digit display
byte SegPins[] = {2,3,4,5,6,7,8};  // 7-segment display pins in the order,{a,b,c,d,e,f,g,dp}
int ultrasonicTriggerPin = 12;
int ultrasonicEchoPin = 13;
```

```
int redPin = 9;
int yellowPin = 10;
int greenPin = 11;

int carCount = 0;  // Variable to keep track of car count

void setup() {
  S.begin(COMMON_CATHODE, 1, CommonPins, SegPins, 1);

  pinMode(redPin, OUTPUT);
  pinMode(yellowPin, OUTPUT);
  pinMode(greenPin, OUTPUT);

  pinMode(ultrasonicTriggerPin, OUTPUT);
  pinMode(ultrasonicEchoPin, INPUT);

  digitalWrite(redPin, HIGH);

  // Initialize the display showing current car count
  displayNumber(carCount);
}

void loop() {
  // Check distance to detect cars passing
  int distance = getDistance();
  if (distance < 10) {
    carCount++; // Increment car count when a car is detected
    displayNumber(carCount); // Update display with new car count
    delay(1000); // Delay to avoid multiple counting for the same car
  }

  // Adjust green light duration based on car count
  if (carCount >= 2) {
    greenLight(10000); // Green light duration set to 10 seconds if there are 2 or more cars
  } else {
    greenLight(5000); // Green light duration set to 5 seconds if there is 1 car
  }
}

void greenLight(int duration) {
  digitalWrite(greenPin, HIGH);
  digitalWrite(yellowPin, LOW);
```

```
  digitalWrite(redPin, LOW);
 delay(duration); // Duration of green light
 digitalWrite(greenPin, LOW);
}

void displayNumber(int num) {
 S.setNumber(num);
 S.refreshDisplay();
 delay(20);
}

int getDistance() {
 // Triggering ultrasonic sensor
 digitalWrite(ultrasonicTriggerPin, LOW);
 delayMicroseconds(2);
 digitalWrite(ultrasonicTriggerPin, HIGH);
 delayMicroseconds(10);
 digitalWrite(ultrasonicTriggerPin, LOW);

 // Reading echo pulse
 long duration = pulseIn(ultrasonicEchoPin, HIGH);

 // Calculating distance in cm
 int distance = duration * 0.034 / 2;

 return distance;
}
```

- ## Results:-

  - When observing the system in action, you'll notice that the signal timings change dynamically based on the flow of traffic.
  - Segments with heavier traffic will have longer green signal durations to accommodate the vehicles waiting there.
  - Conversely, segments with lighter traffic will have shorter green signal durations.

We have attached google drive link to simulated video :-

https://drive.google.com/drive/folders/1u_KqdOeeTbtPLrWHiv9JNJu7c9IhBT6A

- ## Conclusion :-

- We simulated turtlebot3 using Ros and Gazebo and simulated its navigation , perception and control algorithms .

- The traffic control system efficiently regulates traffic flow by dynamically adjusting green light durations facilitated by ultrasonic sensor integration.

- ## References:-

- https://emanual.robotis.com/docs/en/platform/turtlebot3/simulation/#gazebo-simulation

- https://www.instructables.com/Traffic-Signal-Using-Arduino-and-Ultrasonic-Sensor/